

Secure and Globally Distributed Static Website on AWS Using Terraform

Student: Mahmoud Kanaan
Matriculation number: 92133250
Course: DLBSEPCP01_E

Problem and objective

The task is to publish a simple static webpage for visitors located around the world. The solution must remain available without depending on a single server, respond with low latency, accommodate increases in request volume, use HTTPS, keep its origin private, and be reproducible through Infrastructure as Code. The objective is therefore to deploy one self-contained index.html file from a private Amazon S3 bucket and distribute it globally through Amazon CloudFront. The implementation deliberately focuses on cloud architecture rather than web-application complexity.

Architecture and request flow

The architecture contains a global user, one CloudFront distribution, CloudFront Origin Access Control (OAC), one private S3 bucket, and the index.html object. A visitor requests the CloudFront domain over HTTPS. CloudFront redirects any HTTP request to HTTPS and checks its edge cache. When the object is cached, the nearest suitable edge location returns it directly. When CloudFront must contact the origin, OAC signs the request with AWS Signature Version 4. OAC was selected because AWS recommends it for S3 origins and it allows the bucket policy to restrict access to the designated CloudFront distribution, preventing users from bypassing CloudFront through an S3 URL (Amazon Web Services, n.d.-d). The policy permits only s3:GetObject when AWS:SourceArn matches the created distribution.

Selected services and quality requirements

Amazon S3 was selected because the website consists of one object and needs durable managed storage rather than a server. S3 Standard redundantly stores objects across at least three Availability Zones and is designed for 99.99% availability, making it appropriate for the highly available static origin (Amazon Web Services, n.d.-b). Amazon CloudFront was selected because it caches static content at a worldwide network of edge locations and routes viewers to a low-latency location, improving global delivery, availability, and transfer performance (Amazon Web Services, n.d.-e). It also provides HTTPS, compression, and managed caching. PriceClass_All prioritizes the worldwide latency requirement over restricting delivery to lower-cost regions. Both services handle changing request volumes without fixed server capacity or a manually managed autoscaling group. Security is based on multiple controls. All four S3 Public Access Block settings are enabled, ACLs are disabled through BucketOwnerEnforced, and the object is encrypted at rest using SSE-S3 with AES-256. S3 versioning supports recovery from unintended object replacement. CloudFront uses its default TLS certificate for the distribution domain, redirects viewers to HTTPS, and signs private-origin requests through OAC. No credentials are stored in source code, Terraform output, Git, or documentation.

Terraform and alternatives considered

Terraform was selected because it defines infrastructure in human-readable configuration that can be versioned, reused, reviewed, and applied consistently throughout the infrastructure lifecycle (HashiCorp, n.d.). It defines the bucket, ownership controls, Public Access Block, encryption, versioning, HTML object, OAC, distribution, and restricted bucket policy. The bucket name combines the project name with the authenticated AWS account ID for global uniqueness. filemd5() detects webpage changes, while the provider lock file makes dependency selection reproducible. The same reviewed configuration can create the environment with terraform apply and remove it with terraform destroy. force_destroy is acceptable only for this temporary student project. EC2 and a load balancer were rejected because EC2 supplies virtual servers that require an operating system and application maintenance, while Elastic Load Balancing exists to route traffic across registered compute targets (Amazon Web Services, n.d.-a, n.d.-c). This project has neither server-side processing nor multiple targets: S3 stores the object directly and CloudFront handles delivery. ECS, Docker, Lambda, API Gateway, databases, Route 53, a custom domain, ACM, AWS WAF, React, Node.js, backend code, and CI/CD were likewise unnecessary. The smaller architecture reduces cost, permissions, attack surface, and operational work while meeting the stated goals.

Conclusion

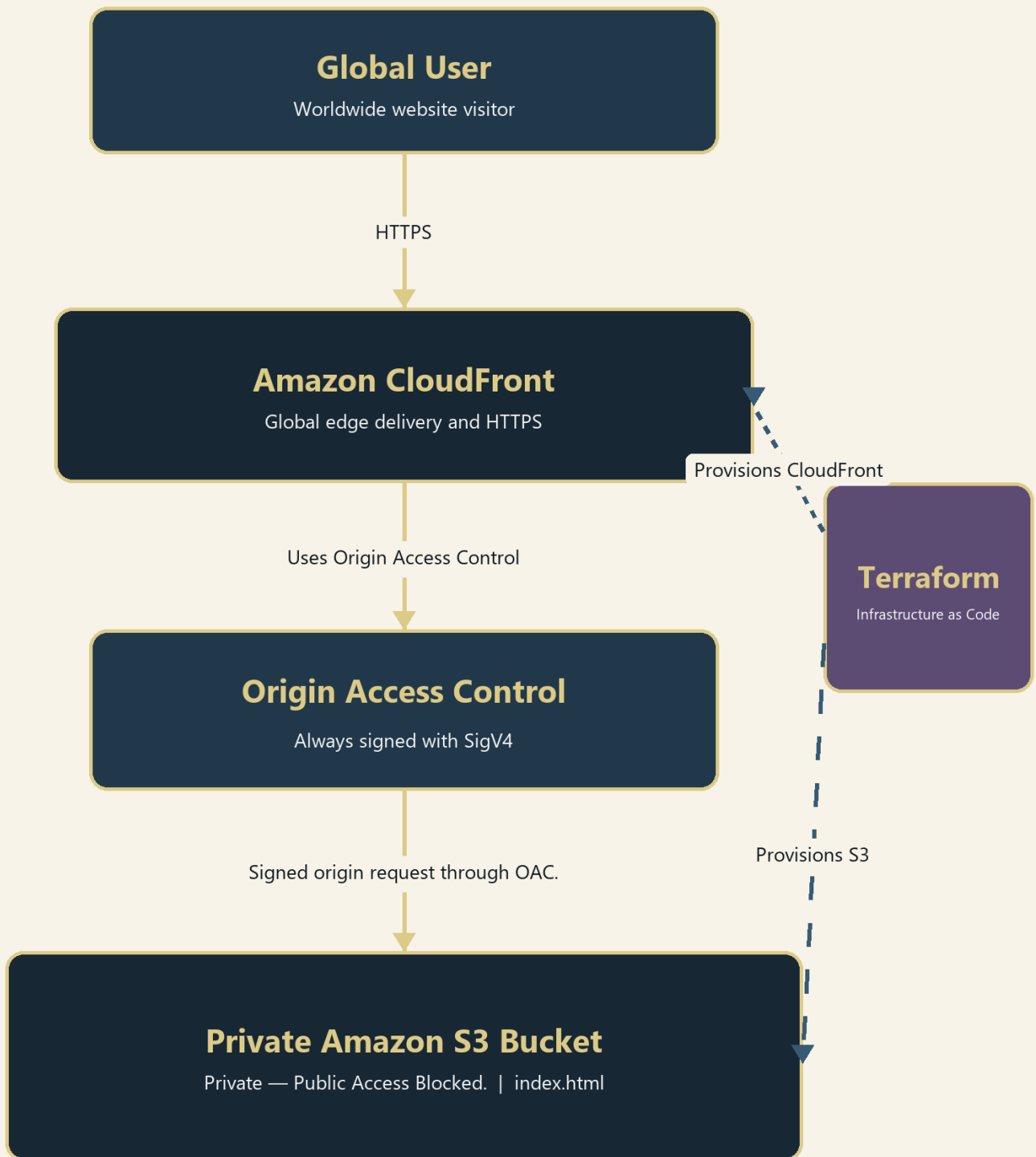
The design uses the smallest suitable set of managed AWS services. CloudFront supplies secure worldwide delivery and caching, while a private S3 origin stores the single webpage. OAC and the source-restricted bucket policy prevent origin bypass, and Terraform makes the deployment understandable, repeatable, testable, and removable after assessment.

References

Amazon Web Services. (n.d.-a). Amazon EC2 instances. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/Instances.html>
Amazon Web Services. (n.d.-b). Data protection in Amazon S3. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/DataDurability.html>
Amazon Web Services. (n.d.-c). How Elastic Load Balancing works. <https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/how-elastic-load-balancing-works.html>
Amazon Web Services. (n.d.-d). Restrict access to an Amazon S3 origin. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html>
Amazon Web Services. (n.d.-e). What is Amazon CloudFront? <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html>
HashiCorp. (n.d.). What is Terraform? <https://developer.hashicorp.com/terraform/intro>

Architecture Diagram

AWS Static Website Architecture



Solid arrows: viewer/origin request flow Dashed arrows: Terraform provisioning